

TravelWatch AI: Flight and Train Ticket Price Prediction with Autonomous Booking Agent*

1st Ruolan Chen 5th Alina Kapanova 2nd Lena Park 3rd Erica Tang 4th Seungyeon Kwon

Abstract—Flight and train ticket prices vary significantly with timing, route, and demand, yet travelers lack tools to determine optimal purchase timing. Prior work [8], [15] shows ML can predict prices but remains limited to passive forecasting, while commercial tools lack automation and global price normalization, leading to systematic overpayment in a \$100+ billion market. TravelWatch AI addresses this gap as an end-to-end ML system for real-time price tracking, prediction, and autonomous booking.

We implement two core approaches: Ridge/Lasso Regression for price prediction and Logistic Regression vs. KNN for Buy/Wait classification, with anomaly detection via Z-score and IQR. Evaluation uses MSE, R^2 , F_1 , AUC-ROC, and Precision@K. We expect Ridge to achieve $R^2 > 0.85$ and Logistic Regression to reach $F_1 > 0.75$, consistent with prior work [10]. This system extends ML from prediction to action, enabling automated, cost-efficient decision-making and providing a generalizable agentic framework for time-sensitive pricing domains.

I. INTRODUCTION

Airfare pricing is highly opaque, with prices for the same itinerary varying by over 50% within short time windows due to timing, availability, and carrier dynamics [1]. As a result, travelers often overpay and face a high cognitive burden from manual price tracking. In a \$100+ billion market, even small inefficiencies translate into billions in losses, despite average U.S. fares of \$384 in 2024 [17]. Prior work explores ML-based fare prediction [8], [15], but existing systems remain limited to passive monitoring, providing alerts without optimizing purchase timing or automating decisions [6], [8]. This gap is increasingly recognized in industry and media, highlighting the need for automated, data-driven purchasing tools [11]–[13].

We model fare dynamics using regression (Ridge/Lasso) for price prediction and classification (Logistic Regression, KNN) for Buy/Wait decisions, with anomaly detection (Z-score, IQR) to flag irregular prices. These methods are well-suited, as regression captures price trends [10], [15], while classification performs effectively on high-dimensional decision tasks [1]. Prior work has focused on prediction, and commercial tools (e.g., Hopper, Google Flights) provide forecasts but require manual action, while agent-based approaches lack real-world integration [8], [9]. Our key contribution is a unified, agentic system that integrates prediction, decision-making, and autonomous execution—including cross-currency normalization and custom URL monitoring—addressing this gap.

The proposed pipeline begins with data exploration and preprocessing on the EaseMyTrip, Expedia, and BTS datasets, including visualization, cleaning, encoding, normalization, outlier filtering, and programmatically generated Buy/Wait

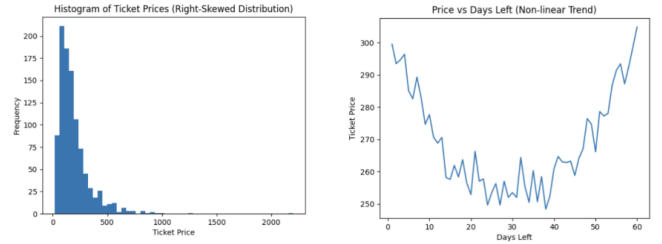


Fig. 1. Histogram of ticket prices (x-axis: price in USD, y-axis: frequency) showing right skew, motivating log transform (left) and line plot of price vs. days_left (x-axis: days until departure, y-axis: price in USD), showing a non-linear decline (right)

labels. It then trains from-scratch NumPy models for price prediction (Linear, Polynomial, Ridge, Lasso), classification (Logistic Regression, KNN), and anomaly detection (Z-score, IQR). These methods are well-suited, as regression captures price trends [10], [15] and classification performs effectively on high-dimensional decision tasks [1]. Our key contribution is integrating prediction with automated execution into a unified, agentic system [8], [9].

Models are evaluated using MSE, MAE, and R^2 (regression), F_1 and AUC-ROC (classification), and Precision@K and Recall@K (anomaly detection). The best models are selected via cross-validation and deployed in a Streamlit application with a FastAPI backend, enabling users to track routes, receive forecasts and Buy/Wait recommendations, and optionally trigger automated booking via the Amadeus API [14]. Expected results include Ridge achieving $R^2 > 0.85$ and Logistic Regression achieving $F_1 > 0.75$. This work extends ML from prediction to action, enabling automated decision-making in time-sensitive pricing domains and generalizing to applications such as ticketing and resource allocation. While improving efficiency, it raises concerns around transparency, user trust, and the impact of automated agents [9], highlighting the need for interpretability and user control.

II. BACKGROUND

A. Comparisons to Prior Work

Tziridis et al. [15] evaluate multiple ML models, achieving ~88% accuracy and identifying key features such as departure lead time and carrier, which inform our baseline and feature selection. Groves and Gini [8], [9] frame purchase timing as an agent problem, achieving 75.3% accuracy but without booking integration. Abdella et al. [1] note reliance on single-source

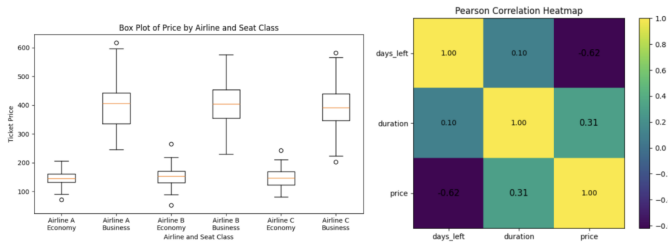


Fig. 2. Box plot of ticket price by airline and seat class (x-axis: airline/class categories, y-axis: price in USD) and Pearson correlation heatmap (color scale: correlation coefficient in range [-1, 1])

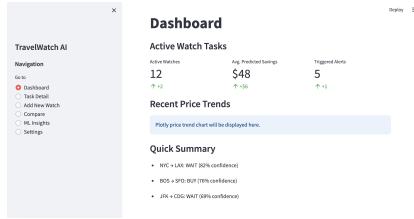


Fig. 3. Screenshot of the Dashboard page showing price trends and summary statistics (values displayed in USD).

data, motivating our multi-source approach. Kalampokas et al. [10] report up to 99% accuracy on large-scale data, guiding our target of $R^2 > 0.85$, while Etzioni et al. [6] highlight the importance of timing decisions without modern scalability. Wang et al. [18] improve prediction via time-series methods but do not address decision automation.

B. Common Knowledge and Disputes

There is broad consensus that booking lead time is the strongest predictor of airfare prices, with fares decreasing as departure approaches up to a threshold and rising sharply near departure [8], [15]. This non-linear pattern motivates our use of polynomial features. Key disputes remain regarding model complexity. While Kalampokas et al. [10] show deep learning models outperform classical approaches on high-variance routes, Tziridis et al. [15] find comparable performance from simpler models on structured datasets, highlighting trade-offs between complexity, interpretability, and data requirements. Another unresolved issue is the gap between prediction and action: existing systems assume users act optimally on forecasts, despite evidence of suboptimal decisions under uncertainty [6], [9]. To address this, we adopt efficient, interpretable models and extend prior work by integrating prediction with automated decision execution in a unified system.

C. Knowledge Gaps

Prior work has focused on fare prediction using single-source data and supervised models, with limited integration into actionable systems [1], [15]. Three gaps motivate TravelWatch AI: lack of cross-currency comparison, absence of autonomous booking execution, and no support for monitoring non-GDS sources via arbitrary URLs. We address these by

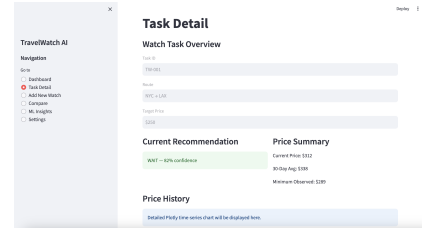


Fig. 4. Screenshot of the Task page.

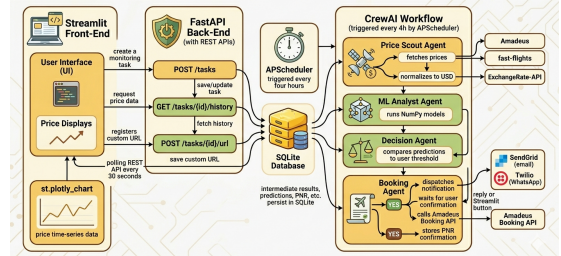


Fig. 5. System architecture diagram illustrating data flow between Streamlit front end, FastAPI back end, CrewAI agents, and external APIs.

integrating multi-source data, cross-currency normalization, prediction, Buy/Wait classification, anomaly detection, and automated execution into a unified pipeline. Based on prior work, we expect Ridge Regression to achieve $R^2 \sim 0.87-0.91$ [10], while Logistic Regression matches or exceeds KNN on F_1 with lower latency, improving both predictive performance and decision efficiency.

III. END-TO-END ML PIPELINE

A. Offline Development Plan and Back-End

Offline development follows a batch learning paradigm, where models are trained on fixed historical datasets in Jupyter Notebooks rather than updated from streaming data. It includes data exploration, preprocessing, feature engineering, and from-scratch training of regression, classification, and anomaly detection models. Experiments use controlled splits, cross-validation, and hyperparameter tuning, evaluated with MSE, MAE, R^2 , F_1 , and AUC-ROC. Final models are refined offline and exported to the Streamlit application for real-time inference, with in-app experiments limited to system-level evaluation. The back-end handles data collection, preprocessing, and execution of trained models for price prediction, Buy/Wait decisions, and anomaly detection. It connects user inputs (route, date, target price) to real-time inference when a watch task is created or updated. Based on the results, it returns price trends and recommendations and can trigger automated actions such as notifications or booking via the Amadeus API.

1) Data Collection, Exploration and Processing:

Primary dataset. We use the EaseMyTrip Flight Price Prediction dataset [3], released on Kaggle in 2022. It contains 300,261 records with 11 features, including airline (categorical), source/destination city (categorical), departure time bin (categorical), number of stops (numerical), flight duration (numerical), seat class (binary: Economy/Business),

and `days_left` (numerical: days until departure). The target variable is ticket price in INR (continuous, numerical). Ground truth price labels are provided directly, and no annotation is required. Dataset URL: <https://www.kaggle.com/datasets/shubhambathwal/flight-price-prediction>. Additionally, we use the Flight Prices dataset (Expedia, Kaggle, 2022) [5] for time-series feature engineering and the U.S. BTS DB1B survey [16] for historical route-level fare baselines. Real-time prices are retrieved from the Amadeus Travel API (sandbox, unlimited free calls) and fast-flights [2], an open-source Google Flights scraper. Exchange rates are fetched hourly from ExchangeRate-API [7].

Buy/Wait labels. For classification, we generate labels programmatically: a booking on day t is labeled *Buy* (1) if the price on day t is below 110% of the minimum price observed over the following 14 days; otherwise, it is labeled *Wait* (0). This follows the labeling approach of Groves and Gini [8].

Data exploration.

Figure 1 presents a histogram of ticket prices showing right skew and justifying log transformation and a line plot of price vs. days left confirming the expected non-linear decline. Figure 2 shows a box plot of price by airline and seat class and a Pearson correlation heatmap identifying days left ($r = -0.62$) and flight duration ($r = 0.31$) as the strongest numerical predictors.

Preprocessing. Missing values (<2% of rows) are dropped to avoid bias from imputation. Categorical features (airline, city, time bin) are one-hot encoded, and the binary class feature is encoded as Economy = 0 and Business = 1. Numerical features (`days_left`, `duration`, `price`) are Z-score normalized ($\mu = 0$, $\sigma = 1$) to ensure comparable scales and stable optimization, excluding the bias term. Outlier fares below \$20 or above \$5,000 (USD equivalent) are removed using IQR filtering. Polynomial features (degree 2) are generated via `combinations_with_replacement` to capture non-linear relationships. Exploratory analysis shows right-skewed price distributions and non-linear temporal trends, motivating log transformation and feature expansion. Correlation analysis identifies `days_left` and `duration` as key predictors, guiding feature selection. These steps follow established practices [1], [10], [15] and improve model stability, interpretability, and performance.

2) *Methods and Model Training:* **Task 1 — Price Regression.** We implement and compare four regression models from scratch, well-suited for airfare prediction due to their strong performance on structured tabular data and interpretability [10], [15].

(i) *Linear Regression:* Minimizes $\mathcal{L} = \|X\mathbf{w} - \mathbf{y}\|^2$ with closed-form solution $\mathbf{w} = (X^\top X)^{-1}X^\top \mathbf{y}$. This corresponds to the maximum likelihood estimator under Gaussian noise and provides an unbiased, minimum-variance linear estimator (Gauss–Markov theorem). It serves as a baseline and performs competitively on structured airfare datasets [15].

(ii) *Polynomial Regression:* Augments features with degree-2 interaction terms before applying linear regression. This corresponds to linear regression in a higher-dimensional fea-

ture space, enabling approximation of non-linear functions via basis expansion and capturing the observed price–lead-time relationship [8].

(iii) *Ridge Regression (L2):* Minimizes $\|X\mathbf{w} - \mathbf{y}\|^2 + \alpha\|\mathbf{w}\|^2$ with solution $\mathbf{w} = (X^\top X + \alpha I)^{-1}X^\top \mathbf{y}$. Ridge introduces bias to reduce variance, addressing multicollinearity and improving generalization via the bias–variance trade-off, particularly in high-dimensional settings from one-hot encoding [10].

(iv) *Lasso Regression (L1):* Minimizes $\|X\mathbf{w} - \mathbf{y}\|^2 + \alpha\|\mathbf{w}\|_1$, solved via coordinate descent. The L1 penalty induces sparsity, performing implicit feature selection and improving interpretability in high-dimensional spaces [1].

Inputs: feature vector $\mathbf{x}$ = [`days_left`, `duration`, `stops`, `airline_onehot`, `class`, ...].

Output: predicted fare $\hat{y} \in \mathbb{R}^+$.

Task 2 — Buy/Wait Classification. We implement and compare two classifiers:

(i) *Logistic Regression:* $P(y=1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$. Derived from the Bernoulli likelihood, it maximizes log-likelihood via convex optimization, ensuring a unique global optimum. Its probabilistic output enables calibrated confidence estimates for decision-making [8].

(ii) *KNN:* Computes Euclidean distance and predicts via majority voting among the k nearest neighbors. As a non-parametric method, it makes minimal distributional assumptions and approximates the Bayes optimal classifier with sufficient data, capturing local structure effectively [15].

Inputs: price deviation from predicted value, recent price slope, anomaly flag, `days_left`, seat class. **Output:** binary label $\in \{0, 1\}$ with confidence score.

Task 3 — Anomaly Detection. We use Z-score and IQR methods for outlier detection.

Z-score: $z = (x - \mu)/\sigma$, which standardizes data and flags statistically rare events.

IQR: $[Q_1 - 1.5 \cdot \text{IQR}, Q_3 + 1.5 \cdot \text{IQR}]$, a robust, non-parametric method resistant to skewed distributions and heavy tails [1].

Inputs: historical price series per route. **Output:** binary anomaly flag.

Agentic AI component (CrewAI). We use CrewAI [4] to orchestrate four agents: a *Price Scout Agent*, *ML Analyst Agent*, *Decision Agent*, and *Booking Agent*, extending prior agent-based approaches with end-to-end ML-driven execution.

3) *Model Evaluation: Metrics.* For regression: $\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$; $\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$; $R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$. For classification: $F_1 = \frac{2PR}{P+R}$ and AUC-ROC. For anomaly detection: Precision@K and Recall@K .

These metrics capture complementary aspects of performance. For regression, MSE penalizes large errors—critical in cost-sensitive settings—while MAE provides interpretable average deviation, and R^2 measures explained variance for comparison with prior work [10], [15]. For classification, F_1 balances precision and recall, reflecting the trade-off between premature booking and missed opportunities, while AUC-ROC evaluates ranking robustness under varying thresholds and class imbalance [1], [8]. For anomaly detection, Precision@K

and Recall@ K focus on identifying top- K extreme price deviations, prioritizing ranking quality over overall accuracy, as commonly used in retrieval settings [1].

Experimental Setup and Evaluation. These metrics also help identify over- and underfitting by comparing train and test performance. Models are trained using an 80/20 train-test split with 5-fold cross-validation. Hyperparameters are tuned as follows: $\alpha \in \{0.01, 0.1, 1, 10\}$ for Ridge/Lasso; polynomial degree $\in \{1, 2\}$; learning rate $\eta \in \{10^{-3}, 10^{-2}, 10^{-1}\}$ and iterations $\in \{100, 500, 1000\}$ for Logistic Regression; and $k \in \{3, 5, 7, 11\}$ (Euclidean distance) for KNN. Anomaly thresholds include $|z| > \{2, 2.5, 3\}$ and IQR multipliers $\in \{1.5, 2.0\}$. Experiments include model comparison, learning curves, and error analysis on large residuals and misclassified Buy/Wait decisions, with ablation studies assessing polynomial features and anomaly signals. Overfitting is mitigated via regularization, cross-validation, and feature selection, while underfitting is addressed through feature expansion and model comparison. This setup ensures reproducibility and systematic evaluation relative to prior work.

4) *Model Deployment:* Models are selected based on trade-offs between predictive performance, stability, and inference efficiency. For regression, models are ranked by test-set R^2 , with MSE as a secondary criterion. If Lasso performs within 2 percentage points of Ridge, Ridge is preferred for its greater stability and lower variance under multicollinearity. For classification, models are evaluated using F_1 and AUC-ROC. If KNN is within 3 percentage points of Logistic Regression in F_1 , Logistic Regression is selected due to lower inference latency and better scalability for real-time decision-making, reflecting a trade-off between local accuracy (KNN) and efficiency (Logistic). Final selection prioritizes accuracy, robustness, and computational cost under real-time constraints. Trained model parameters (NumPy arrays) are serialized to JSON and loaded at runtime by the Streamlit application and CrewAI ML Analyst Agent.

B. Front-End Application Using Streamlit

Target population and UI layout (2 pts + 2 pts). Primary users are price-sensitive travelers—students, frequent flyers, and international commuters—seeking to minimize fares without continuous monitoring. Secondary users include course reviewers evaluating model performance. The application uses a sidebar-based navigation structure (built from the PAML Streamlit template [14]) with six pages: (1) Dashboard, (2) Task Detail, (3) Add New Watch, (4) Compare, (5) ML Insights, and (6) Settings. Figures 3 and 4 show screenshots of the Dashboard and Task Detail pages, web app demo at <https://github.com/alinak78/Travel-AI-Agent>

User inputs and outputs (4 pts). The *Add New Watch* page includes inputs for origin/destination (IATA autocomplete), date, target price, currency selection (170 currencies via ExchangeRate-API), and notification preferences (email/WhatsApp). Outputs include a price trend chart (Plotly), a BUY/WAIT badge with confidence, and summary statistics (current, minimum, maximum, 30-day average, per-

centile rank). On the *ML Insights* page, users select a model via a radio button; the app displays learning curves, a confusion matrix heatmap, an ROC curve, and feature importance plots from offline experiments.

Front-end/back-end connection (10 pts). Figure 5 illustrates the system architecture. The Streamlit front end communicates with a FastAPI back end via REST APIs: POST /tasks creates a task; GET /tasks/{id}/history retrieves price time series; POST /tasks/{id}/url registers custom URLs. APScheduler triggers the CrewAI workflow every four hours: the Price Scout Agent collects data (Amadeus, fast-flights) and normalizes it via ExchangeRate-API; the ML Analyst Agent runs NumPy models; the Decision Agent evaluates thresholds; and the Booking Agent sends notifications (SendGrid/Twilio) or executes booking via the Amadeus API upon user confirmation. Results are stored in SQLite, and the Streamlit app polls the API every 30 seconds to update outputs without full reload.

IV. RISK AND MITIGATION

Risk 1 — Amadeus sandbox data realism. Sandbox data may not reflect real market dynamics. Mitigation: supplement with fast-flights (Google Flights data) and validate models on Kaggle datasets before live deployment.

Risk 2 — Low ML accuracy. If $R^2 < 0.80$, we will (a) add higher-degree polynomial features, (b) tune regularization, and (c) incorporate high-signal features such as weekday and holiday proximity [15].

Risk 3 — Custom URL scraping failures. Websites may block requests or change structure. Mitigation: restrict to a validated whitelist (e.g., Ryanair, Ctrip, Viator) and fall back to manual entry if extraction fails.

Risk 4 — Timeline constraints. The 12-week timeline is tight. Mitigation: decouple ML and infrastructure work for parallel progress, with a feature freeze at Week 9 to allow testing and reporting.

V. EXPECTED OUTCOMES

Outcome 1. Ridge Regression is expected to achieve $R^2 > 0.85$ on the EaseMyTrip test set, outperforming Linear Regression by at least 8 percentage points, consistent with prior work [10].

Outcome 2. Logistic Regression is expected to achieve $F_1 > 0.75$ for Buy/Wait classification with sub-millisecond latency, matching or slightly outperforming KNN while being significantly faster for real-time deployment.

Outcome 3. The Streamlit application will demonstrate an end-to-end workflow: a user submits a route, the system generates a 30-day forecast with confidence intervals, and triggers automated notification and mock booking when the target price is met.

VI. TEAM MEMBER CONTRIBUTIONS

The team is split across ML modeling (A–B), backend, data infrastructure (C–D), and front-end/UI (E). Writing tasks align with a member’s technical contributions as shown in the table.

TABLE I
TEAM RESPONSIBILITIES

Who	Technical	Writing
A	NumPy; Linear/Poly/Ridge/Lasso; learning curves; MSE/R ² evaluation	Sections 4.1.1 (data), 4.1.2 (regression)
B	NumPy; Logistic/KNN; Z-score/IQR; F1/AUC/Precision@K	Sections 4.1.2 (classification), 4.1.3 (evaluation)
C	Data pipelines (Amadeus, fast-flights, FX); APScheduler; SQLite	Sections 4.1.1 (preprocessing), 5 (risk)
D	FastAPI; CrewAI agents; Twilio/SendGrid; Amadeus booking	Sections 4.1.4 (deployment), 4.2 (back-end)
E	Streamlit UI; Plotly charts; Custom URL (BeautifulSoup + Claude API); Figma wireframes	Sections 1, 2, 4.2 (UI/UX), 6

REFERENCES

- [1] Jumana Abdella, Nizar Zaki, Khaled Shuaib, and Farhan Khan. Airline ticket price and demand prediction: A survey. *Journal of King Saud University - Computer and Information Sciences*, 33, 2021.
- [2] AWeirdDev. fast-flights: Google flights scraper. GitHub, 2023.
- [3] Shubham Bathwal. Flight price prediction dataset (easemytrip). Kaggle, 2022.
- [4] CrewAI Inc. Crewai: Framework for orchestrating role-playing autonomous ai agents. GitHub, 2024.
- [5] dilwong. Flight prices (expedia). Kaggle, 2022.
- [6] Oren Etzioni, Rattapoom Tuchinda, Craig A. Knoblock, and Alexander Yates. To buy or not to buy: Mining airfare data to minimize ticket purchase price. In *9th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2003.
- [7] ExchangeRate-API. Free and pro currency exchange rates api, 2024.
- [8] William Groves and Maria Gini. An agent for optimizing airline ticket purchasing. In *International Conference on Autonomous Agents and Multi-Agent Systems (AAMAS)*, 2013.
- [9] William Groves and Maria Gini. Airline ticket purchasing: An agentic approach with machine learning. *ACM Transactions on Intelligent Systems and Technology*, 7(1), 2015.
- [10] Theodoros Kalampokas et al. A holistic approach on airfare price prediction using machine learning, deep learning, and quantum machine learning. *IEEE Access*, 11:6073–6087, 2023.
- [11] Melissa Kravitz. How does dynamic pricing for airlines affect your travels? *Forbes*, 2024.
- [12] Washington Post. Flight deal experts spot a surprise upcharge: Flying solo. *The Washington Post*, 2025.
- [13] Alison Sider. Airlines are charging higher fares and are confident you’ll pay up. *The Wall Street Journal*, 2025.
- [14] A. Taylor. PamI final project streamlit templates. GitHub, 2026.
- [15] Konstantinos Tziritidis, Theodoros Kalampokas, George Papakostas, and Konstantinos Diamantaras. Airfare prices prediction using machine learning techniques. In *25th IEEE European Signal Processing Conference (EUSIPCO)*. IEEE, 2017.
- [16] U.S. Bureau of Transportation Statistics. Airline origin and destination survey (db1b). TranStats, 2024.
- [17] U.S. Bureau of Transportation Statistics. 2024 annual average domestic air fare. BTS, 2025.
- [18] X. Wang et al. Flight price prediction using machine learning and time series models. *Journal of Air Transport Management*, 2020.